Lookup Table (LUT) in VLSI

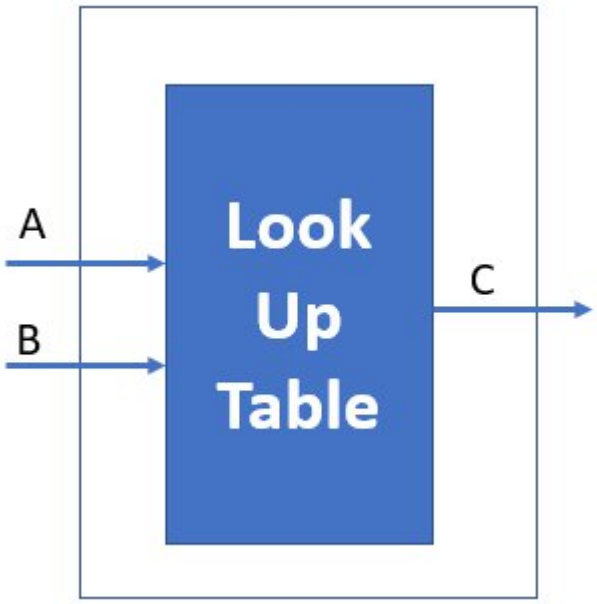
- Memory-based logic implementation
- Widely used in FPGA and ASIC
- Implements Boolean functions using stored values

Memory-based way to implement digital logic.
Instead of building a Boolean function using logic gates, function's truth table is stored in memory, and the inputs "**look up**" correct output

output = table[input]

What is a LUT?

LUT stores truth table values
Inputs act as address
Output is selected stored bit



Inputs A and B are the address pins and C is t

Every time your address pins are changing pointing at a different address entry and they are “reading out” result

LUT Size

- N-input LUT requires 2^N memory bits

2-LUT → 4 bits

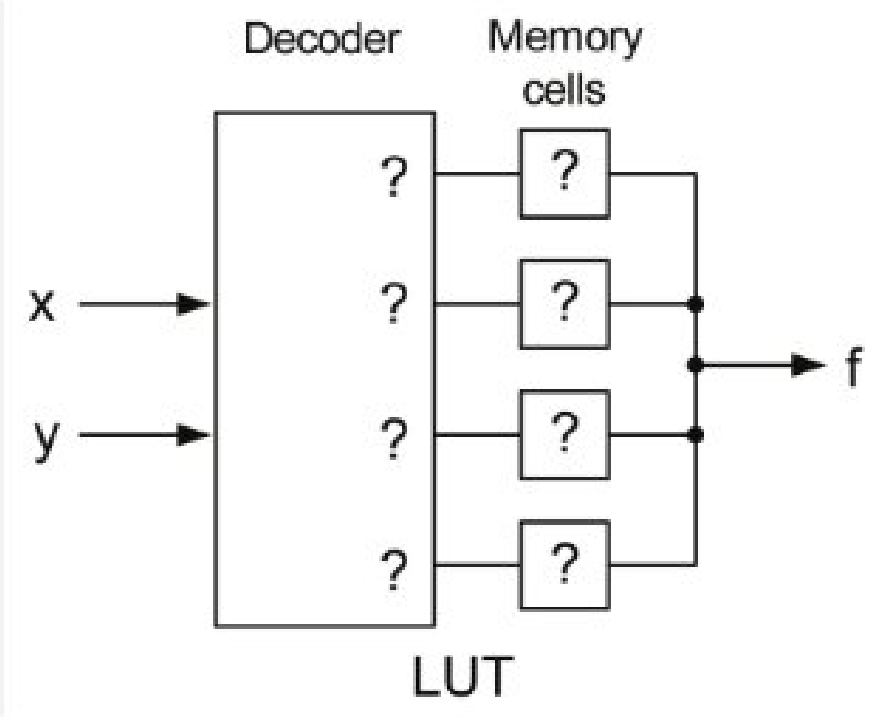
3-LUT → 8 bits

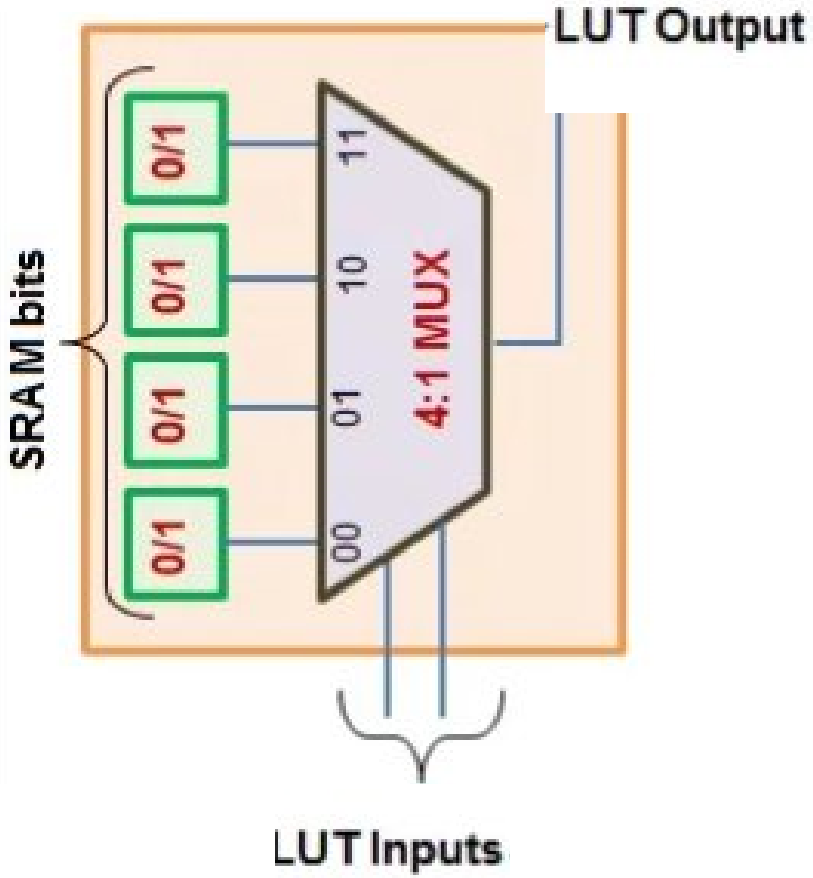
6-LUT → 64 bits

Realizes Boolean functions by mapping input combinations to pre-stored output values.

A	B	Address	Output
0	0	00	f(0,0)
0	1	01	f(0,1)
1	0	10	f(1,0)
1	1	11	f(1,1)

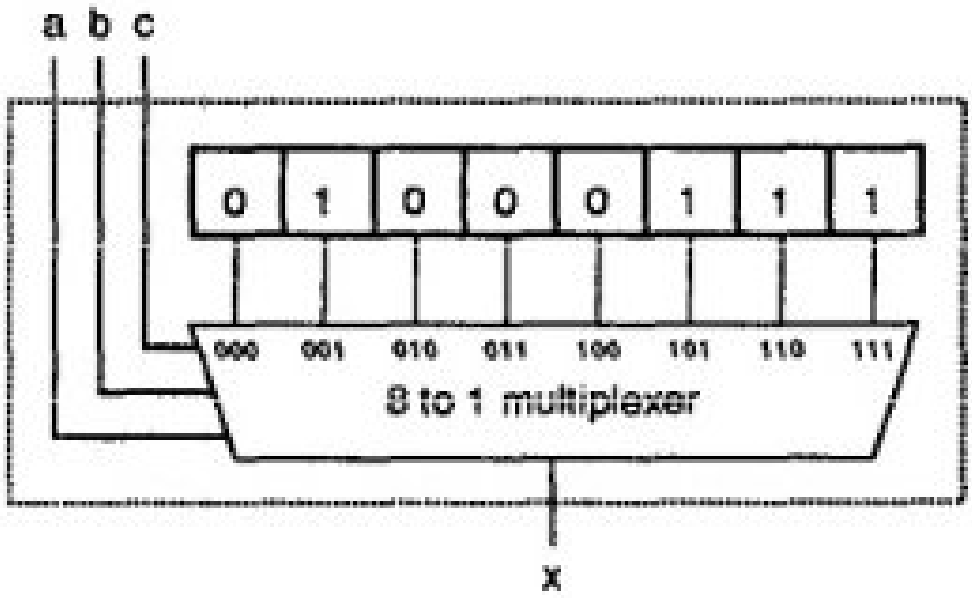
Decoder-Mux based LUT





Inputs → Decoder → Wordlines
|
Memory Cells (SRAM/ROM)
|
Output

Inputs → MUX Tree → Output
▲
Stored bits


a b c
0 1 0 0 0 1 1 1
000 001 010 011 100 101 110 111
8 to 1 multiplexer
x

2-Input LUT (XOR)

- Function: A ⊕ B
- Truth Table: 00→0, 01→1, 10→1, 11→0
- Implement using 4-bit memory

```
module lut2_xor (
    input  wire A,
    input  wire B,
    output wire Y
);
    // 2^2 = 4 bits LUT
    wire [3:0] lut_mem = 4'b0110;
    assign Y = lut_mem[{A, B}];
endmodule
```

3-Input LUT (Majority Function)

$$F=AB+BC+AC$$

4-Input LUT (Custom Boolean Function)

$$F=(A\&B) \oplus (C|D)$$

LUT Using ROM Style (ASIC-Friendly)

```
module lut_rom (
   input  [2:0] addr,
   output reg  Y
);
   always @(*)
begin
      case (addr)
         3'b000: Y = 0;
         3'b001: Y = 1;
         3'b010: Y = 0;
         3'b011: Y = 1;
         3'b100: Y = 1;
         3'b101: Y = 0;
         3'b110: Y = 1;
         3'b111: Y = 0;
      endcase
   end
endmodule
```

LUT Using output Style (FPGA-Friendly)

```
module lut_with_ff (
   input  clk,
   input  [2:0] addr,
   output reg  Y
);
   reg [7:0] lut_mem =
8'b11010010;

   always @(posedge clk)
      Y <= lut_mem[addr];
endmodule
```

Array in Verilog

An array is a collection of elements of the same data type, accessed using an index.

Vector wire [7:0] mem 8-bit wide signal

1D Array wire [7:0] mem [0:15]; 16 elements → 0 to 15 Each element is 8 bits

```
mem[0] = 8'hAA;    // write
data = mem[5];    // read
```

2D Array wire [7:0] mem [0:3][0:3]; 4 × 4 matrix Each entry = 8 bits

```
matrix[1][2] = 8'd15;
data = matrix[row][col];
```

LUT based Full Adder

```
module lut_full_adder (
   input  A,
   input  B,
   input  Cin,
   output Sum,
   output  Cout
);
   // 3-input LUTs → 8 entries
 wire [7:0] lut_sum  = 8'b01101001;
wire [7:0] lut_cout = 8'b00010111;

assign Sum  = lut_sum[{A,B,Cin}];
assign Cout = lut_cout[{A,B,Cin}];
endmodule
```

```
module lut_fa(
   input  A, B, Cin,
   output Sum, Cout
);
   // Each entry = {Cout, Sum}
   reg [1:0] lut_mem [0:7];

   initial begin
      lut_mem[0] = 2'b00; // 000
      lut_mem[1] = 2'b01; // 001
      lut_mem[2] = 2'b01; // 010
      lut_mem[3] = 2'b10; // 011
      lut_mem[4] = 2'b01; // 100
      lut_mem[5] = 2'b10; // 101
      lut_mem[6] = 2'b10; // 110
      lut_mem[7] = 2'b11; // 111
   end

   assign {Cout, Sum} = lut_mem[{A,B,Cin}];
endmodule
```

LUT-Based 4-Bit Ripple Carry Adder

```
module lut_adder_4bit (
   input  [3:0] A, B,
   input  Cin,
   output [3:0] Sum,
   output Cout
);
   wire [4:0] C;
   assign C[0] = Cin;
```

```
genvar i;
   generate
      for (i=0; i<4; i=i+1) begin : LUT_FA
         lut_full_adder FA (
            .A   (A[i]),
            .B   (B[i]),
            .Cin (C[i]),
            .Sum (Sum[i]),
            .Cout(C[i+1])
         );
      end
   endgenerate

   assign Cout = C[4];
endmodule
```

LUT based 2-bit Multiplier

```
module lut_mul_2x2 (
   input  [1:0] A,
   input  [1:0] B,
   output [3:0] P
);
   // 16-entry LUT, each entry is 4-bit product
   reg [3:0] lut [0:15];

   initial begin
      lut[ 0] = 0;   // 0x0
      lut[ 1] = 0;
      lut[ 2] = 0;
      lut[ 3] = 0;
      lut[ 4] = 0;
      lut[ 5] = 1;
      lut[ 6] = 2;
      lut[ 7] = 3;
      lut[ 8] = 0;
      lut[ 9] = 2;
      lut[10] = 4;
      lut[11] = 6;
      lut[12] = 0;
      lut[13] = 3;
      lut[14] = 6;
      lut[15] = 9;
   end

   assign P = lut[{A,B}];
endmodule
```

3-bit × 3-bit LUT-Based Multiplier

```
module lut_mul_3x3 (
   input  [2:0] A,
   input   [2:0] B,
   output [5:0] P
);
   reg [5:0] lut [0:63];
   integer i, j;
   initial begin
      for (i = 0; i < 8; i = i + 1)
         for (j = 0; j < 8; j = j + 1)
            lut[{i[2:0], j[2:0]}] = i * j;
   end
   assign P = lut[{A,B}];
endmodule
```

LUT for Square function

$Y = x^2$

```
module square_lut (x,y);
   input  [3:0] x;
   output [7:0] y;

reg [7:0] table [0:15];

integer i;
initial
begin
   for(i=0;i<16;i=i+1)
      table[i] = i*i;
end

assign y = table[x];
endmodule
```

Applications

- FPGA logic
- DSP & AI
- Cryptography
- Hardware obfuscation

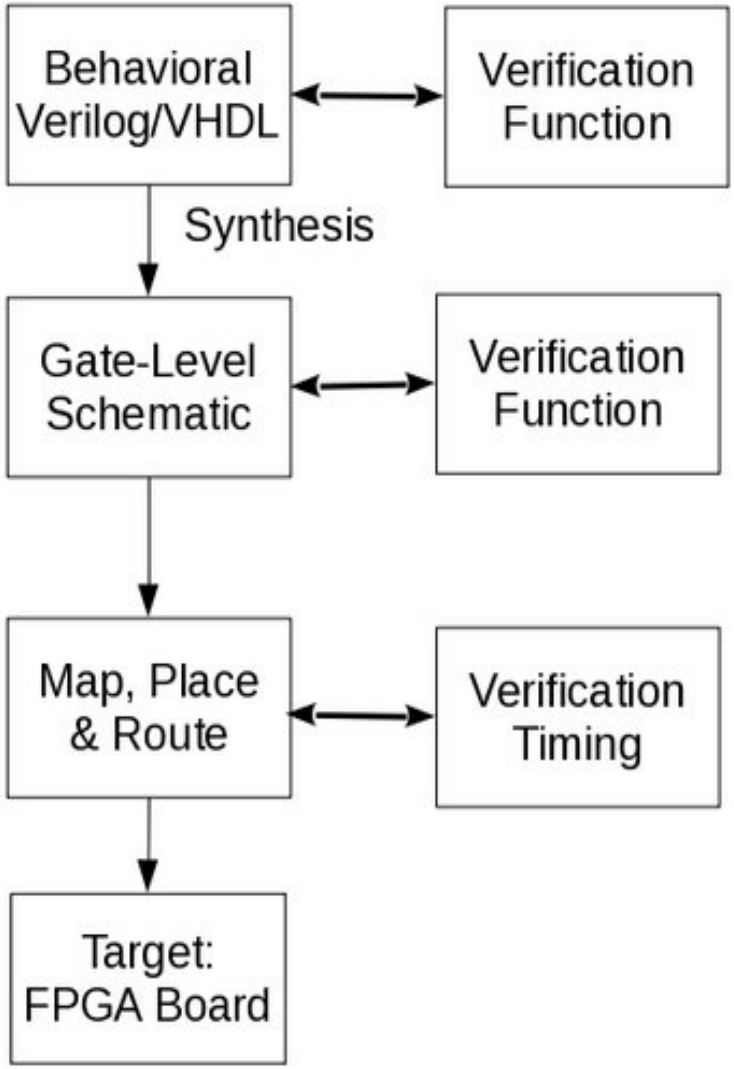
✅ Fast logic evaluation
✅ Regular layout (easy routing)
✅ Good for complex Boolean functions
✅ Reconfigurable (FPGA)
✅ Power-efficient for repeated operations

ASIC Vs FPGA

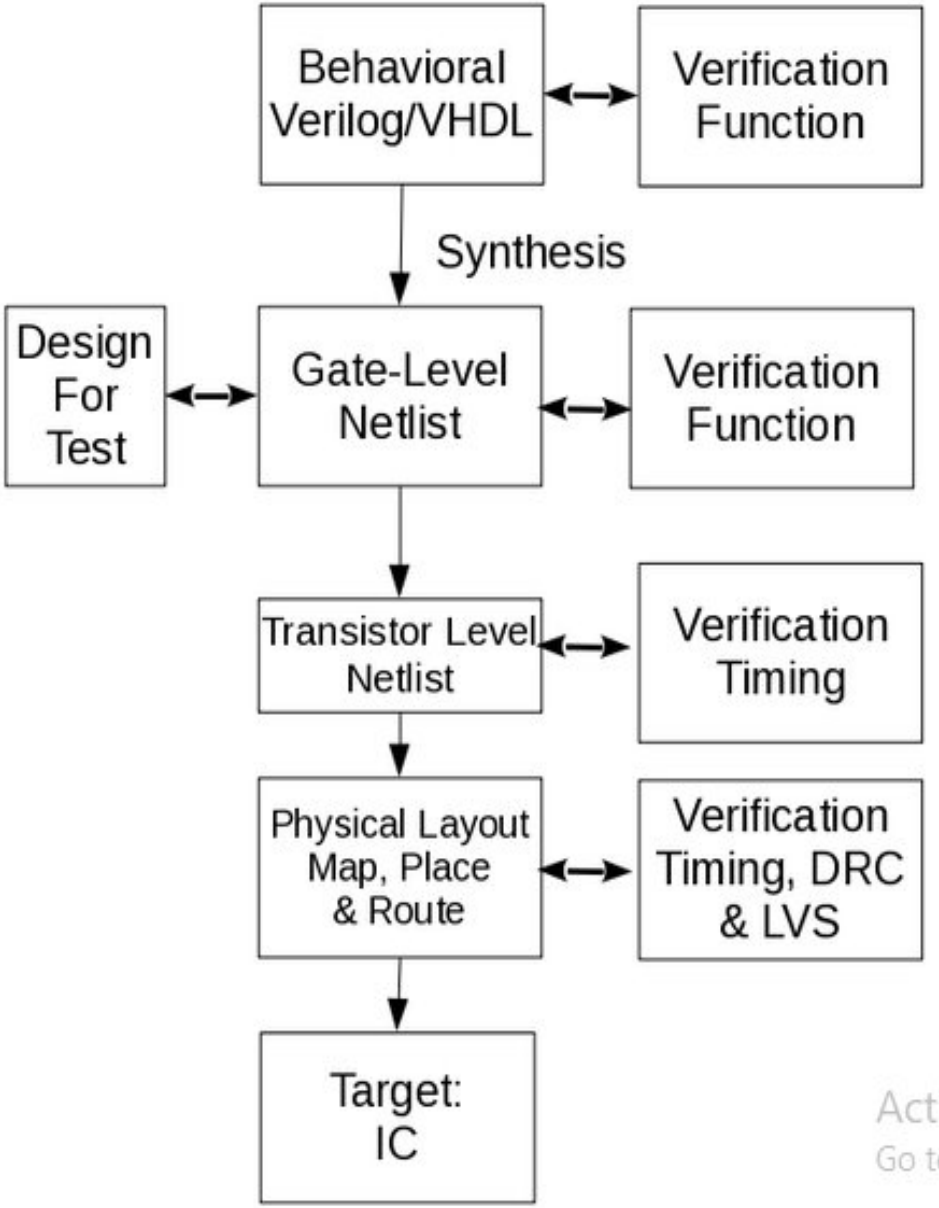
ASIC anf FFPGA are specialized IC
ASICs designed for high-volume, high performance applications
FPGAs offering flexibility, reconfigurability, and rapid prototyping

	FPGA	ASICs
Full Form	Field Programmable Gate Array	Application-Specific Integrated Circuit
Design Process	Simple design process.	Long and complex design process.
Expenses	There are no non-recurring expenses.	It is expensive as it involves the cost of circuit design and mask design.
Termed as	Faster "time-to-market" product.	Longer "time-to-market" product.
Speed	Slower than ASIC.	Fast.
Reusability and Flexibility	Reusable and flexible.	Not reusable and not flexible.
Wastage	Un-avoidable	No wastage of hardware.
Best suited	When the required numbers are less.	When the required numbers are large.

FPGA design flow

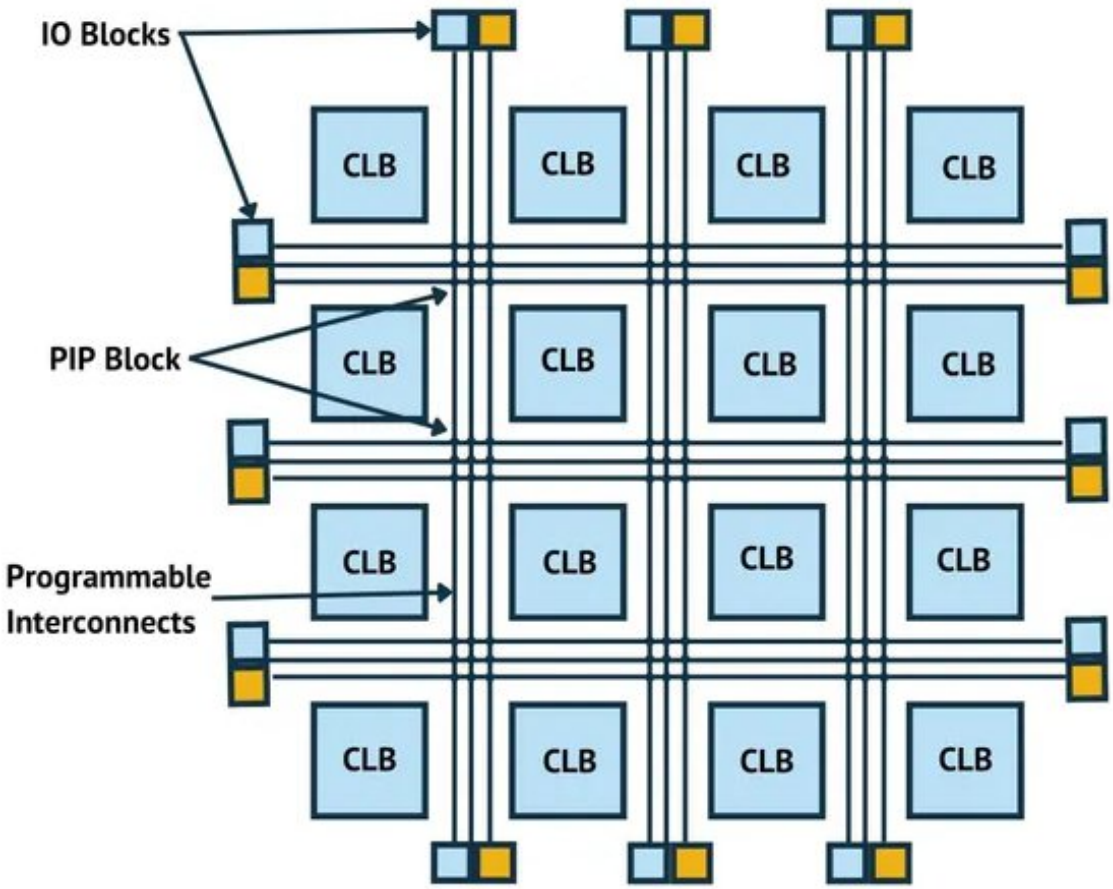


Custom ASIC design flow



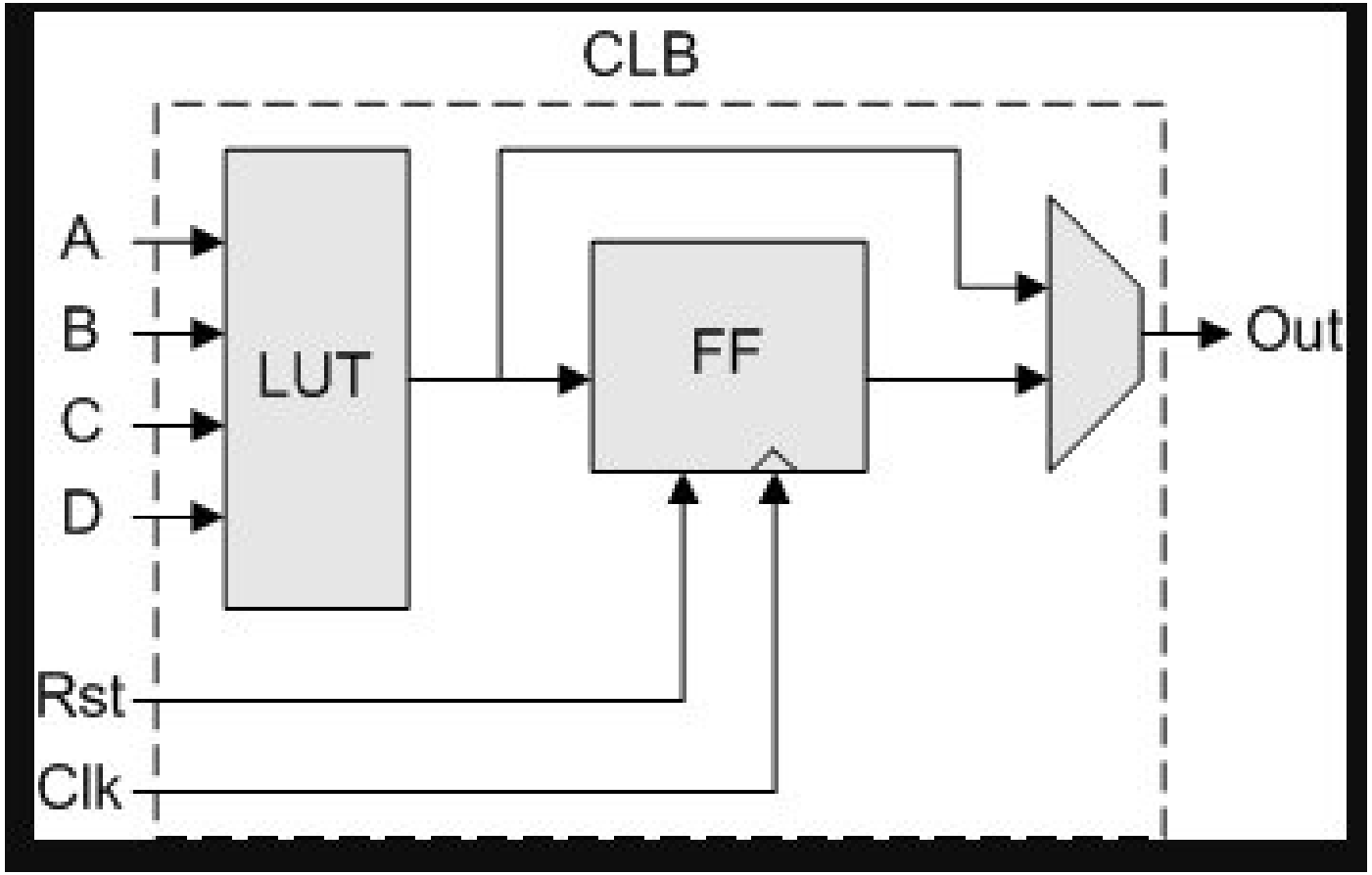
Architecture

FPGA are semiconductor devices that can be **programmed and reprogrammed** after manufacturing to implement digital logic functions.



Components of an FPGA

1. Configurable Logic Blocks (CLBs): basic building blocks of an FPGA containing Look-Up Tables (LUTs), flip-flops, and dedicated arithmetic units. CLBs configured to perform various combinational and sequential logic functions



2. Programmable Interconnect Points (PIPs): are programmable switches and wiring resources

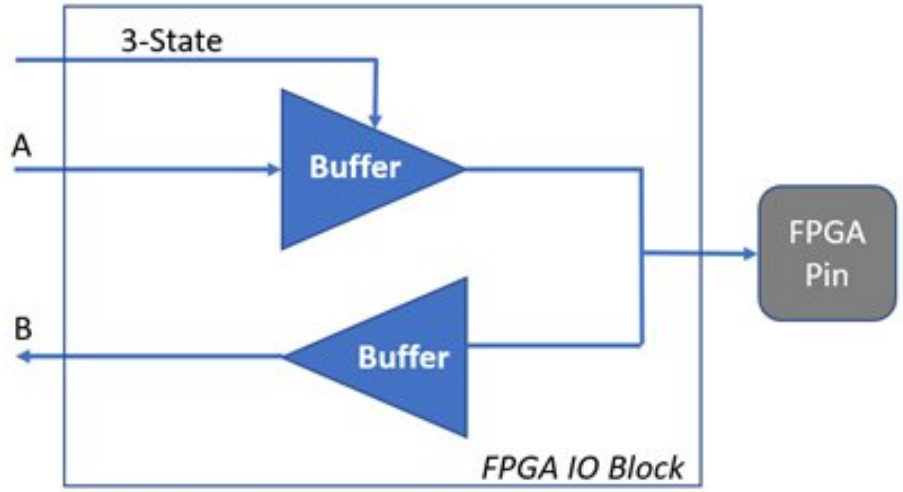
Connect CLBs and other components within the FPGA.

PIPs allow for flexible routing and interconnection of signals between different elements on the chip.

3. Input/Output Blocks (IOBs): Interface points between FPGA and external devices or other components on the board.

IOBs are configurable enables the FPGA to communicate with peripherals.

IOB select FPGA pin input, output or tri-state.

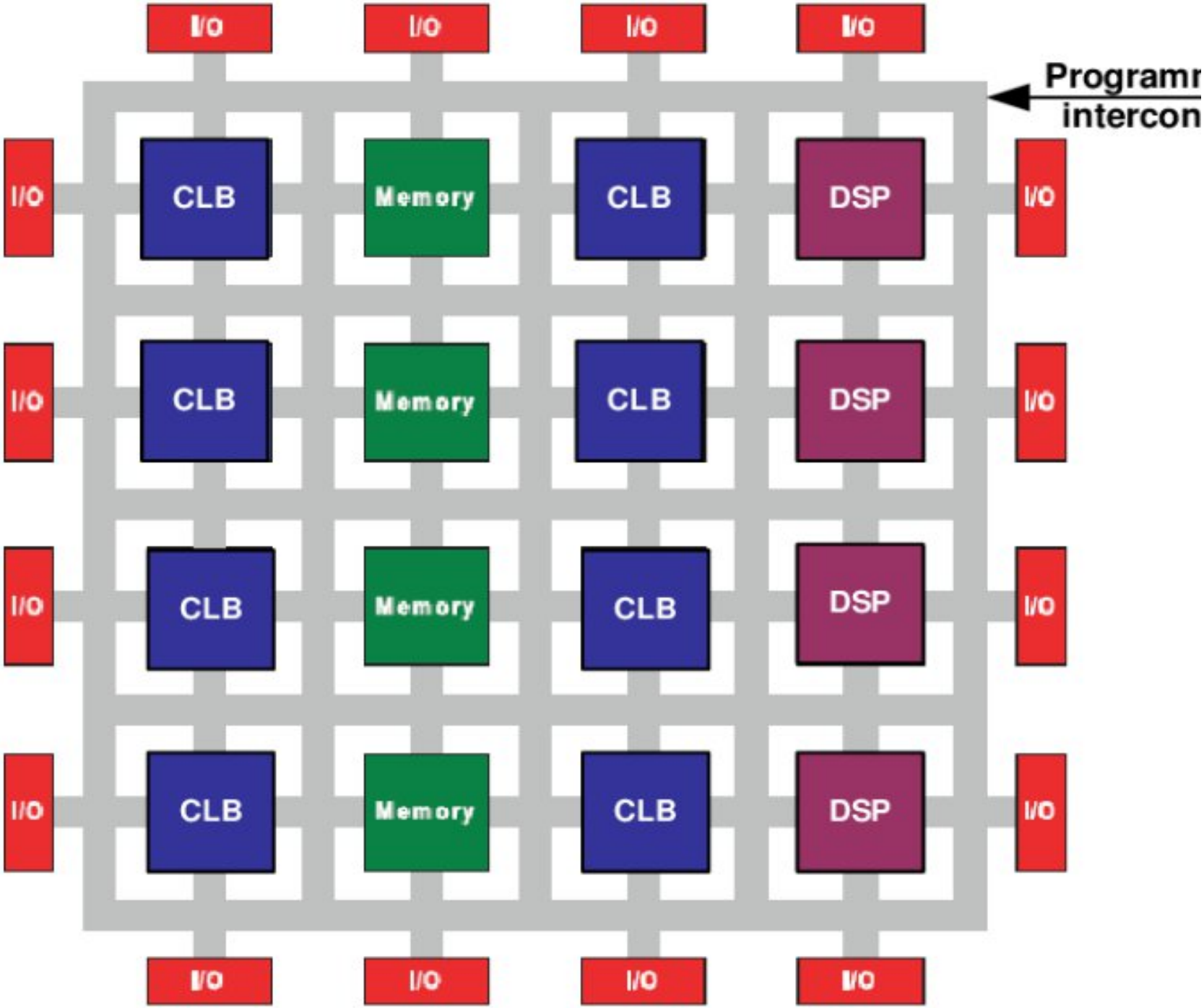


4. Block RAM (BRAM): is on-chip memory available on an FPGA, used for storing data and implementing memory functions.
BRAMs are faster than off-chip memory and can be accessed with lower latency.

5. Configuration memory: flash memory that stores the configuration bitstream. The bitstream is loaded into the FPGA at startup or when reprogramming required.

6. Digital Signal Processing (DSP) Blocks: specialized hardware blocks dedicated to performing arithmetic operations, multiplication and accumulation, at high speeds. DSP blocks are commonly used in signal processing and filtering applications.

7. Clock management resources: Phase-Locked Loops (PLLs) and Digital Clock Managers (DCMs) used to generate, distribute, and manage clock signals within FPGA. These resources help maintain precise timing and synchronization across the device.


I/O
CLB
Memory
DSP
Programm
intercon

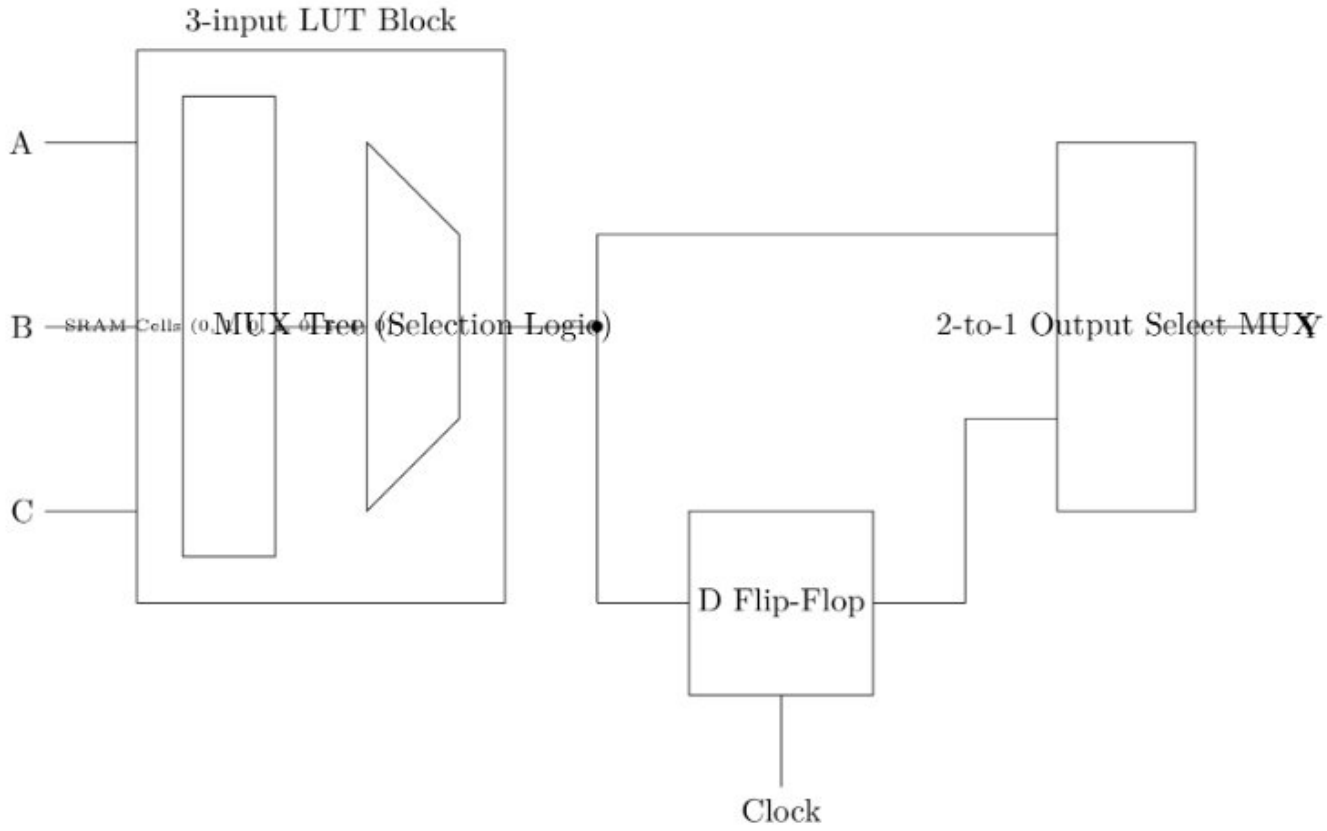
Example

f(A,B,C) = (A.B)XOR C

primary component within a CLB is LUT (Look-Up Table), stores pre-computed results of a Boolean function.

A	B	C	AB XOR C
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

3-input LUT (LUT3), 2^3 = 8 memory cells
Memory Content = [0, 1, 0, 1, 0, 1, 1, 0]

D Flip-Flop (DFF): An edge-triggered register.
The input D receives the LUT result.

Signal from a dedicated SRAM Configuration Cell located within the FPGA's configuration memory.
0 => Combinational Mode, Y is driven directly by the LUT.
1 => Sequential Mode, Y is driven by the D Flip-Flop (synchronized to the clock).